Alaude

A desktop AI assistant for your computer — not your browser tab

Ahmed Al-Sayadi · 2026-04-18

Electron 33 · Node 18+ · MIT

Why we're here

1. **Background** — the Claude Code CLI leak, and what it did (and didn't) reveal
2. **What Alaude is** — and what it is *not*
3. **Architecture** — three processes, three concerns
4. **Key engineering calls** — DNS, worker, IPC, tool streaming
5. **Features that matter** — Spaces, local models, tool calling, OODA
6. **Demo**
7. **Roadmap**

Part 1 — Background

The Claude Code CLI "leak"

Claude Code ships as a **minified Node bundle** installed via `npm i -g @anthropic-ai/claude-code`.

In early 2025 the community **de-minified the bundle** and published the unpacked source on GitHub. Not a security breach — just a reverse-engineering of what was shipped to every developer's machine.

What it exposed:

- The system prompts Anthropic uses for the agent
- The tool schema (`Read`, `Edit`, `Write`, `Bash`, `Grep`, `Glob`, `TodoWrite`, `Agent`, …)
- The loop: plan → tool call → observe → repeat
- How sub-agents, hooks, and skills are wired

What it did *not* expose: model weights, server-side routing, Anthropic's infra.

Why the leak mattered

It proved that **Claude Code's "magic" is mostly prompt engineering + a tool loop** around the Anthropic API — not some proprietary runtime.

That knowledge spawned an ecosystem:

- **Clones** — dozens of "open source Claude Code" repos
- **Forks** — per-domain variants (data, devops, security)
- **Agent SDKs** — everyone now ships their own loop

Alaude is **not** one of those clones. It uses the *lesson* of the leak (a good tool loop is simple) but applies it to a different form factor: **the desktop**.

Claude Code (CLI) vs Alaude (desktop)

Dimension	Claude Code CLI	Alaude Desktop
Surface	Terminal REPL	Electron window with chat UI
Provider	Anthropic only	Anthropic, OpenAI, Google, xAI, Moonshot, Qwen, GLM, **Ollama (local)**
Primary user	Developers in a repo	Anyone — Health, Finance, Legal, RE, Edu, Marketing, General
Personalization	`CLAUDE.md` + skills	**Spaces** — domain-tuned system prompts + quick actions
Local inference	No	Yes — in-app Ollama catalog (Qwen 3.6, Gemma 4, Llama 3.3, DeepSeek R1)
Observability	None (client-side)	**OODA loop** — local health score + diagnosis
Tool loop	Rich, battle-tested	Minimal: `read_file`, `write_file`, `list_directory`, `run_command`, `open_in_browser`, `start_dev_server`
License / source	Proprietary, minified	**MIT, source-available**

In one sentence

> Claude Code is a **CLI for coders**. Alaude is a **desktop app for everyone else** — with a coder mode bolted on.

Part 2 — Architecture

Three processes, three concerns

```
                                IPC                                JSON/stdio
┌──────────────────────┐   ──────────>   ┌──────────────────────┐   ──────────────────>   ┌──────────────────┐
│  Renderer            │                 │  Main (Electron)     │                         │  API Worker      │
│  renderer/           │   <──────────   │  electron/main.js    │   <──────────────────   │  (plain Node)    │
│  index.html          │                 │                      │                         │ api-worker.js    │
└──────────────────────┘                 └──────────────────────┘                         └──────────────────┘

 UI + user events                         IPC routing, file ops,                           Talks to LLMs,
 (12,981 lines, single file)              Ollama mgmt, credentials,                        executes tools,
                                          OAuth PKCE (1,182 lines)                         DNS patch (667)
```

Why three? See next slide — the worker isn't decorative.

Why a separate worker process?

Two hard-won reasons:

1. Electron's `ELECTRON_RUN_AS_NODE` broke HTTPS under VPNs

A user on Astrill VPN couldn't reach `api.openai.com` or `api.anthropic.com`. Electron's bundled Node had a different DNS code path that didn't honor the VPN's resolver correctly.

Fix: the worker runs under the **system `node` binary** with a monkey-patched `dns.lookup` that tries the system resolver first (preserves VPN routing) and falls back to `8.8.8.8` / `1.1.1.1` on timeout.

```
dns.lookup = function patchedLookup(hostname, options, callback) {
  const timeout = setTimeout(() => {            // 3s budget
    publicResolver.resolve4(hostname, ...)      // fall back to 8.8.8.8/1.1.1.1
  }, 3000)
  _origLookup.call(dns, hostname, options, ...) // try system DNS first
}
```

2. The UI doesn't freeze while a 60-second LLM stream chugs through the worker.

IPC protocol

Main ↔️ Worker speak **JSON lines over stdio**:

```
> { id: 42, kind: 'chat', provider: 'anthropic', model: 'claude-sonnet-4-6',
    messages: [...], tools: [...], space: 'health' }

< { id: 42, activity: { phase: 'tool_start', name: 'write_file', args: {...} } }
< { id: 42, activity: { phase: 'tool_end',   name: 'write_file', ok: true } }
< { id: 42, activity: { phase: 'token', text: 'Sure, here ' } }
< { id: 42, result: { content: '...', usage: {...} } }
```

Main demuxes on `id` and forwards each `activity` to the renderer via `webContents.send('tool-activity', ...)`.

The renderer paints live chips — the user sees **something moving within 1 second**, even when the final answer takes 60.

Part 3 — Features

🧭 Spaces — the killer feature

Same chat box, seven different brains.

Built-in Spaces:

- **General**
- **Health** — labs, drug interactions, PHQ-9 / GAD-7, BMI/BMR
- **Finance** — budgets, invoices, P&L, cashflow
- **Real Estate** — ROI, MLS
- **Legal** — NDAs, contract review
- **Education** — lesson plans, quizzes
- **Marketing** — social, SEO, ad copy

Each Space provides:

- Tuned system prompt
- Quick-action buttons
- Placeholder text

🦙 Local models (Ollama) as first-class citizens

- In-app catalog: **Qwen 3.6**, **Gemma 4** (E2B/E4B/26B/31B), **Llama 3.2/3.3**, **DeepSeek R1**
- One-click download with progress bar + cancel
- Installed models appear in the same dropdown as cloud models
- Tool calling **gated per model** — disabled for tiny models (`gemma3:1b`, `llama3.2:1b/3b`, `deepseek-r1`) that can't format tool JSON

Why it matters: the prompt can stay on the machine when it shouldn't leave (health data, financials, contracts).

🛠️ Tool calling

Six tools, scoped to a workspace folder the user picks:

```
read_file            write_file          list_directory
run_command          open_in_browser     start_dev_server
```

The workspace hint pattern: if a tool-capable model is selected but no folder picked, the "Choose folder" button pulses with a one-line nudge. Silent otherwise. Fixes the "why didn't it save my file?" failure mode.

Live activity log in the chat bubble — each tool call renders as a green-bordered pulsing chip:

- 📝 `write_file · src/game.py`
- ⚡ `run_command · python /tmp/game.py`
- ✓ `run_command` (dimmed when complete)
- ✕ failed (red)

📊 OODA loop — the self-observing app

Observe → every interaction writes to `~/.claude/alaude-events.ndjson`
Orient → every 10 outcomes, group by dimension, compute stats
Decide → six priority-ordered rules, **one proposal per batch**
Act → proposal written to `~/.claude/alaude-ux-proposals.md` — **never auto-applied**

The six diagnose rules:

1. High error rate on a provider
2. High retry rate on a space × model pair
3. Quick-action abandonment
4. Provider latency / high model-switch rate
5. Underused quick-actions
6. Healthy fallback

> *Iron law: humans decide anything affecting UX copy.*

OODA in action — real dogfood fixture

From `electron/test-ooda-fixture.js` — 98 synthetic events across 4h:

- Ollama connection storm
- High-retry finance × openai
- Slow Anthropic
- Healthy finish

Bug found by OODA itself: abandonment logic was inverted. It compared each outcome to the **session's last event** — flagging every middle-of-session outcome as abandoned.

Fix: compare to the **next event after this completion** within the 30s window.

Metric	Before	After
Abandon rate	37%	12%
Mean health score	0.30	0.56

Commit: `d18d9aa`.

🔐 Security & privacy posture

- **Credentials:** `~/.claude/.credentials.json`, mode `0600`. Never leave the machine except as outbound provider requests.
- **Telemetry:** 100% local. OODA events and proposals write to disk only.
- **Renderer ↔️ Main:** isolated via `contextBridge` (`electron/preload.js`). No `nodeIntegration: true`.
- **XSS:** markdown renderer escapes HTML entities *before* applying markdown transforms. Health cards and fenced code extracted first so neither gets mangled.
- **Health Space:** system prompt carries a "not medical advice" reminder on every response.

Chat UX — where the polish lives

- Real markdown rendering — headers, lists, bold, inline code, fenced blocks with copy buttons + language tags
- **Smart auto-scroll** — won't yank you down when you scroll up (threshold: 80px)
- Hover-to-copy every message · 📋 Copy / 💾 Save
- Keyboard: `⌘K` focus input · `⌘N` new session · `Esc` close modals
- Voice input via Web Speech API
- File attachments — **PDF, DOCX, XLSX, images, plain text** parsed inline
- Drag-and-drop

Part 4 — Scale and demo

Scale (as of v0.2.62)

File	LOC	Role
`renderer/index.html`	12,981	Entire UI — chat, Spaces, modals, dashboard
`electron/main.js`	1,182	IPC, window mgmt, OAuth, Ollama, credentials
`electron/api-worker.js`	667	Provider routing, tool execution, DNS patch
`electron/ooda.js`	596	Event log, batch analyzer, 6 diagnose rules
`electron/health/*`	—	Lab ranges, drug interactions, PHQ-9/GAD-7

Ships as **unsigned .dmg** (arm64 + x64) on GitHub Releases. Windows/Linux planned.

Demo plan (5 min)

1. **Switch Spaces** — General → Health → paste a fake lab result
2. **Pick a workspace** — show the pulsing hint disappear
3. **Cloud tool call** — Claude writes a tiny pygame, `run_command` runs it, browser opens
4. **Local model** — pull Gemma 4 E2B live from catalog, run the same prompt offline
5. **Open Insights** — show the live health score + the last OODA diagnosis
6. **Proposals file** — `cat ~/.claude/alaude-ux-proposals.md` to show what the loop suggested

Roadmap

Next session (highest value):

- **Token streaming** — `stream: true`, per-token delta → new IPC phase. Tool-activity layer stays on top. New OODA metric: time-to-first-token.

After that:

- Scheduled prompts
- MCP server support (bring the Claude Code tool ecosystem in)
- Signed + notarized macOS builds
- Windows / Linux releases

TL;DR for the audience

- The Claude Code leak taught the industry that **the CLI's magic is a simple tool loop + good prompts.**
- Alaude takes that lesson to the **desktop**, with **domain Spaces**, **multi-provider routing including local**, and a **self-observing OODA loop**.
- It's MIT, it runs fully offline if you want, and your keys + telemetry never leave the machine.

Questions?

GitHub · `alsayadi/alaude-desktop` · MIT